Elec 475

Lab 2 Post Lab

Michael Moser

20349246

2. Architectural Specifications

2.1 Common setup and formulas

All four networks take a 3×64×64 RGB tensor and end in a fully connected (FC) layer with K outputs: K = 10 for CIFAR-10 and K = 200 for Tiny ImageNet. CIFAR-10's 32×32 images are resized to 64×64 before entering the network, while Tiny ImageNet images are already 64×64. Each channel is normalized with the dataset's mean and standard deviation, and the tables below use K = 10.

The spatial output size of every convolution and pooling layer, with kernel size k, stride s and padding p, is:

$$H_{out} = \left\lfloor \frac{H_{in} + 2p - k}{s} \right\rfloor + 1$$

A convolution with C_{in} input and C_{out} output channels, and an FC layer with n_{in} inputs and n_{out} outputs, have these parameter counts (the +1 is the bias):

$$P_{conv} = (k^2 C_{in} + 1)\, C_{out}, \qquad P_{FC} = (n_{in} + 1)\, n_{out}$$

Compute is counted in multiply-accumulate operations (MACs) per image; one MAC equals two floating-point operations (FLOPs):

$$\mathrm{MAC}_{conv} = k^2\, C_{in}\, C_{out}\, H_{out}\, W_{out}, \qquad \mathrm{MAC}_{FC} = n_{in}\, n_{out}$$

The receptive field r_l of layer l is the square of input pixels that one of its output units sees. The jump j_l is the distance, in input pixels, between neighbouring output units. Starting from $r_0 = j_0 = 1$ at the input, each convolution or pooling layer updates them as:

$$r_l = r_{l-1} + (k_l - 1)\, j_{l-1}, \qquad j_l = j_{l-1}\, s_l$$

Activations, normalization and dropout leave r and j unchanged, and an FC layer sees the whole image. For CIFAR-10, a receptive field of r pixels in the 64×64 input covers r/2 pixels of the original 32×32 image. Per-layer MACs in the tables count convolution and FC layers only; the FLOP totals in Section 2.6 come from the thop profiler used by train.py.

2.2 Base LeNet-5

LeNet-5 (LeCun et al., 1998) has two convolution stages and three FC layers, with 338,486 parameters and 3.59 M MACs (7.17 MFLOPs) per image. Both convolutions use 5×5 filters with stride 1 and no padding, and each is followed by tanh and non-overlapping 2×2 average pooling with stride 2. The network has no normalization and no dropout.

The feature map shrinks as follows: Conv1 gives (64 − 5)/1 + 1 = 60, AvgPool1 gives (60 − 2)/2 + 1 = 30, Conv2 gives (30 − 5)/1 + 1 = 26 and AvgPool2 gives (26 − 2)/2 + 1 = 13. The classifier therefore receives 16 × 13 × 13 = 2,704 features.

Layer	Configuration	Output (C×H×W)	Parameters	MACs	Receptive field (jump)
Input	RGB image	3×64×64	0	0	1 px (1)
Conv1 + tanh	6 filters, 5×5, stride 1, pad 0	6×60×60	456	1,620,000	5 px (1)
AvgPool1	2×2, stride 2	6×30×30	0	0	6 px (2)
Conv2 + tanh	16 filters, 5×5, stride 1, pad 0	16×26×26	2,416	1,622,400	14 px (2)
AvgPool2	2×2, stride 2	16×13×13	0	0	16 px (4)
Flatten	16×13×13 → 2,704	2,704	0	0	
FC1 + tanh	2,704 → 120	120	324,600	324,480	whole image
FC2 + tanh	120 → 84	84	10,164	10,080	whole image
FC3 (logits)	84 → 10	10	850	840	whole image
Total			**338,486**	**3,577,800**	

- **Parameters:** the two convolutions hold only 2,872 parameters (0.8%), e.g. Conv2 = $(5^2 \times 6 + 1) \times 16 = 2{,}416$. The FC layers hold 335,614 (99.2%), and FC1 alone holds $(2{,}704 + 1) \times 120 = 324{,}600$ (95.9%).
- **Compute:** the split reverses. The convolutions apply their filters at every position, so they perform 3.24 M of the 3.58 M MACs (90.6%).
- **Receptive field:** $5 \to 5 + (2 - 1) \times 1 = 6 \to 6 + (5 - 1) \times 2 = 14 \to 14 + (2 - 1) \times 2 = 16$ pixels. A unit in the last feature map sees 16×16 pixels, a quarter of the image width (8×8 pixels of an original CIFAR-10 image), so global context only enters at FC1.
- **K = 200:** FC3 becomes 84 → 200 (17,000 parameters), for a total of 354,636.

2.3 Modernized LeNet

The Modernized LeNet keeps LeNet-5's filter counts and FC widths but changes three things, ending at 446,006 parameters and 4.59 M MACs (9.19 MFLOPs) per image:

1. **ReLU replaces tanh.** ReLU(x) = max(0, x) has a gradient of 1 for every positive input. The tanh gradient, $1 - \tanh^2(x)$, shrinks toward 0 as |x| grows, which slows learning in saturated units.
2. **3×3 max pooling with stride 2 replaces 2×2 average pooling.** Neighbouring windows overlap by one pixel, and each window keeps its strongest response instead of averaging it away.
3. **Both convolutions use padding 2**, so each keeps its input size ("same" padding).

The feature map shrinks as follows: Conv1 gives $(64 + 4 - 5)/1 + 1 = 64$, MaxPool1 gives $\lfloor(64 - 3)/2\rfloor + 1 = 31$, Conv2 gives $(31 + 4 - 5)/1 + 1 = 31$ and MaxPool2 gives $\lfloor(31 - 3)/2\rfloor + 1 = 15$. The classifier therefore receives $16 \times 15 \times 15 = 3{,}600$ features.

Layer	Configuration	Output (C×H×W)	Parameters	MACs	Receptive field (jump)
Input	RGB image	3×64×64	0	0	1 px (1)
Conv1 + ReLU	6 filters, 5×5, stride 1, pad 2	6×64×64	456	1,843,200	5 px (1)
MaxPool1	3×3, stride 2	6×31×31	0	0	7 px (2)
Conv2 + ReLU	16 filters, 5×5, stride 1, pad 2	16×31×31	2,416	2,306,400	15 px (2)
MaxPool2	3×3, stride 2	16×15×15	0	0	19 px (4)
Flatten	16×15×15 → 3,600	3,600	0	0	
FC1 + ReLU	3,600 → 120	120	432,120	432,000	whole image
FC2 + ReLU	120 → 84	84	10,164	10,080	whole image
FC3 (logits)	84 → 10	10	850	840	whole image
Total			**446,006**	**4,592,520**	

- **Parameters:** the convolutions are unchanged at 2,872. All 107,520 extra parameters are in FC1, whose input grew from 2,704 to 3,600 features: (3,600 − 2,704) × 120 = 107,520. The FC layers now hold 99.4% of the parameters, and FC1 alone holds 96.9%.
- **Compute:** the padded, larger feature maps raise convolution MACs from 3.24 M to 4.15 M, which is 90.4% of the total.
- **Receptive field:** 5 → 5 + (3 − 1) × 1 = 7 → 7 + (5 − 1) × 2 = 15 → 15 + (3 − 1) × 2 = 19 pixels. The overlapping pools widen it from 16 to 19 pixels (9.5 pixels of an original CIFAR-10 image).
- **K = 200:** 462,156 parameters.

2.4 Full AlexNet-64

AlexNet-64 scales the 2012 AlexNet design (Krizhevsky et al.) down to 64×64 inputs: five convolution stages with 64, 192, 384, 384 and 256 filters, then three FC layers with dropout. It has 6,608,842 parameters (6,611,402 with BatchNorm) and 218.35 M MACs per image. Each unit of its last feature map sees 97×97 pixels, more than the whole 64×64 image.

Compared with the 224×224 original, Conv1 uses 5×5 filters with stride 2 instead of 11×11 with stride 4, and the hidden FC layers are 1,024 units wide instead of 4,096. The rest follows the original: ReLU after every convolution and hidden FC layer, overlapping 3×3 max pooling with stride 2 after stages 1, 2 and 5, and dropout (p = 0.5) before FC1 and FC2.

The feature map shrinks as follows: Conv1 gives $\lfloor(64 + 4 - 5)/2\rfloor + 1 = 32$, MaxPool1 gives $\lfloor(32 - 3)/2\rfloor + 1 = 15$, Conv2 keeps 15, MaxPool2 gives $\lfloor(15 - 3)/2\rfloor + 1 = 7$, Conv3 to Conv5 keep 7 and MaxPool3 gives $\lfloor(7 - 3)/2\rfloor + 1 = 3$. The classifier therefore receives 256 × 3 × 3 =

2,304 features. The table shows the variant without normalization; LRN and BatchNorm layers do not change any output size.

Layer	Configuration	Output (C×H×W)	Parameters	MACs	Receptive field (jump)
Input	RGB image	3×64×64	0	0	1 px (1)
Conv1 + ReLU	64 filters, 5×5, stride 2, pad 2	64×32×32	4,864	4,915,200	5 px (2)
MaxPool1	3×3, stride 2	64×15×15	0	0	9 px (4)
Conv2 + ReLU	192 filters, 5×5, stride 1, pad 2	192×15×15	307,392	69,120,000	25 px (4)
MaxPool2	3×3, stride 2	192×7×7	0	0	33 px (8)
Conv3 + ReLU	384 filters, 3×3, stride 1, pad 1	384×7×7	663,936	32,514,048	49 px (8)
Conv4 + ReLU	384 filters, 3×3, stride 1, pad 1	384×7×7	1,327,488	65,028,096	65 px (8)
Conv5 + ReLU	256 filters, 3×3, stride 1, pad 1	256×7×7	884,992	43,352,064	81 px (8)
MaxPool3	3×3, stride 2	256×3×3	0	0	97 px (16)
Flatten	256×3×3 → 2,304	2,304	0	0	
FC1 + ReLU	Dropout (p = 0.5), then 2,304 → 1,024	1,024	2,360,320	2,359,296	whole image
FC2 + ReLU	Dropout (p = 0.5), then 1,024 → 1,024	1,024	1,049,600	1,048,576	whole image
FC3 (logits)	1,024 → 10	10	10,250	10,240	whole image
Total			**6,608,842**	**218,347,520**	

The normalization layer is chosen with train.py's –norm flag:

- **None (–norm none):** an identity layer, as in the table above.
- **Local Response Normalization (–norm lrn):** as in the 2012 paper, placed after the ReLU of stages 1 and 2 only, with n = 5 neighbouring channels, α = 10^{-4}, β = 0.75 and k = 2. It has no learned parameters. For channel i at position (x, y) in a layer with N channels (PyTorch divides α by n):

$$b^i_{x,y} = \frac{a^i_{x,y}}{\left(k + \frac{\alpha}{n}\sum_{j=\max(0,\, i-n/2)}^{\min(N-1,\, i+n/2)} \left(a^j_{x,y}\right)^2\right)^{\beta}}$$

- **Batch Normalization (–norm bn):** BatchNorm2d after every convolution and before its ReLU, in all five stages. For each channel, the mean μ_B and variance σ_B^2 are taken over the mini-batch and all spatial positions, and γ and β are learned per channel. This adds 2 × (64 + 192 + 384 + 384 + 256) = 2,560 learned parameters, plus 2,560

running statistics (a mean and a variance per channel) that replace μ_B and σ_B^2 at evaluation time.

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \qquad y = \gamma\,\hat{x} + \beta$$

Breakdown of the network without normalization:

- **Parameters:** the split is nearly even, with 3,188,672 in the convolutions (48.2%) and 3,420,170 in the FC layers (51.8%). The largest single layer is FC1 at (2,304 + 1) × 1,024 = 2,360,320 (35.7%); the largest convolution is Conv4 at (3^2 × 384 + 1) × 384 = 1,327,488.
- **Compute:** the convolutions perform 214.93 M of the 218.35 M MACs (98.4%). Conv2 alone performs 5^2 × 64 × 192 × 15 × 15 = 69.12 M (31.7%).
- **Receptive field:** 5 → 9 → 25 → 33 → 49 → 65 → 81 → 97 pixels, e.g. MaxPool1 gives 5 + (3 − 1) × 2 = 9 and Conv2 gives 9 + (5 − 1) × 4 = 25. From Conv4 on (65 pixels), one unit spans the full 64-pixel width, so the last stages combine features from the whole image.
- **K = 200:** FC3 becomes 1,024 → 200 (205,000 parameters), for a total of 6,803,592 (6,806,152 with BatchNorm).

2.5 PyTorch Model Zoo AlexNet

The baseline is torchvision's alexnet() with only its last layer replaced (4,096 → K). It has 57,044,810 parameters, 8.6× as many as AlexNet-64, but needs only 94.09 M MACs per 64×64 image, because its 11×11 stride-4 first convolution shrinks the image to 15×15 at once.

torchvision implements the single-GPU version of AlexNet (Krizhevsky, 2014): five convolutions with 64, 192, 384, 256 and 256 filters, ReLU, no LRN or BatchNorm, and 3×3 stride-2 max pooling after Conv1, Conv2 and Conv5. An AdaptiveAvgPool2d layer then fixes the map at 6×6, followed by Dropout, 9,216 → 4,096, Dropout, 4,096 → 4,096 and 4,096 → K. The design targets 224×224 inputs, where the last pool already outputs 6×6; ImageNet weights can be loaded with –pretrained.

At 64×64 the feature map shrinks as follows: Conv1 gives $\lfloor(64 + 4 - 11)/4\rfloor + 1 = 15$, MaxPool1 gives 7, Conv2 keeps 7, MaxPool2 gives 3, Conv3 to Conv5 keep 3 and MaxPool3 gives $\lfloor(3 - 3)/2\rfloor + 1 = 1$.

Layer	Configuration	Output (C×H×W)	Parameters	MACs	Receptive field (jump)
Input	RGB image	3×64×64	0	0	1 px (1)
Conv1 + ReLU	64 filters, 11×11, stride 4, pad 2	64×15×15	23,296	5,227,200	11 px (4)
MaxPool1	3×3, stride 2	64×7×7	0	0	19 px (8)

Layer	Configuration	Output (C×H×W)	Parameters	MACs	Receptive field (jump)
Conv2 + ReLU	192 filters, 5×5, stride 1, pad 2	192×7×7	307,392	15,052,800	51 px (8)
MaxPool2	3×3, stride 2	192×3×3	0	0	67 px (16)
Conv3 + ReLU	384 filters, 3×3, stride 1, pad 1	384×3×3	663,936	5,971,968	99 px (16)
Conv4 + ReLU	256 filters, 3×3, stride 1, pad 1	256×3×3	884,992	7,962,624	131 px (16)
Conv5 + ReLU	256 filters, 3×3, stride 1, pad 1	256×3×3	590,080	5,308,416	163 px (16)
MaxPool3	3×3, stride 2	256×1×1	0	0	195 px (32)
AdaptiveAvgPool	output fixed at 6×6	256×6×6	0	0	195 px (32)
Flatten	256×6×6 → 9,216	9,216	0	0	
FC1 + ReLU	Dropout (p = 0.5), then 9,216 → 4,096	4,096	37,752,832	37,748,736	whole image
FC2 + ReLU	Dropout (p = 0.5), then 4,096 → 4,096	4,096	16,781,312	16,777,216	whole image
FC3 (logits)	4,096 → 10	10	40,970	40,960	whole image
Total			**57,044,810**	**94,089,920**	

- **The 1×1 collapse:** MaxPool3 leaves one value per channel (256×1×1). AdaptiveAvgPool2d copies that value into all 36 cells of the 6×6 grid, so FC1's 9,216 inputs contain only 256 distinct numbers (confirmed by running the model). FC1's 37,752,832 weights, 66.2% of the model, therefore act on 256 features, and Conv3 to Conv5 see only 3×3 maps.
- **Parameters:** 2,469,696 in the convolutions (4.3%) and 54,575,114 in the FC layers (95.7%).
- **Compute:** at 64×64 the FC layers perform more work (54.57 M MACs, 58.0%) than the convolutions (39.52 M).
- **Receptive field:** 11 → 19 → 51 → 67 → 99 → 131 → 163 → 195 pixels, e.g. Conv3 gives 67 + (3 − 1) × 16 = 99. The final 195-pixel field is three times the image width, so most of each unit's window is zero padding.
- **K = 200:** 57,823,240 parameters.

2.6 Comparison of the four architectures

Parameters grow from 338 K to 446 K, 6.6 M and 57.0 M across the four networks, but compute does not follow: at 64×64, AlexNet-64 needs 2.3× the MACs of the 8.6× larger Zoo AlexNet.

Property	Base LeNet-5	Modernized LeNet	AlexNet-64	Zoo AlexNet
Conv / FC layers	2 / 3	2 / 3	5 / 3	5 / 3

Property	Base LeNet-5	Modernized LeNet	AlexNet-64	Zoo AlexNet
Activation	tanh	ReLU	ReLU	ReLU
Pooling	2×2 average	3×3 max, stride 2	3×3 max, stride 2	3×3 max, stride 2, then adaptive average to 6×6
Normalization	none	none	none, LRN or BatchNorm	none
Parameters (K = 10)	338,486	446,006	6,608,842 (BN: 6,611,402)	57,044,810
Parameters (K = 200)	354,636	462,156	6,803,592 (BN: 6,806,152)	57,823,240
Share of parameters in FC layers	99.2%	99.4%	51.8%	95.7%
MACs per image	3.59 M	4.59 M	218.35 M (BN: 218.98 M)	94.10 M
FLOPs per image	7.17 M	9.19 M	436.70 M (BN: 437.97 M)	188.20 M
Last feature map	16×13×13	16×15×15	256×3×3	256×1×1, copied to 6×6
Final receptive field	16 px	19 px	97 px	195 px

MACs and FLOPs are per 64×64 image with K = 10, measured with the thop profiler in train.py, which counts LRN as zero operations. Both LeNets keep over 99% of their parameters in FC layers yet spend about 90% of their compute in convolutions. AlexNet-64 splits its parameters roughly evenly and spends 98% of its compute in convolutions, while the Zoo AlexNet at 64×64 spends most of both in its FC layers.

3. Empirical Results and Figures

3.1 Experimental setup

Every model in the master tables was trained for 10 epochs with the same settings, so that differences between rows come from the architecture, normalization or augmentation alone.

Setting	Value
Hardware	Apple M2 MacBook Pro, 8 GB unified memory, PyTorch MPS (GPU) backend
Optimizer	Adam, learning rate 10^{-3}, weight decay 10^{-4}, no learning-rate schedule
Batch size, epochs, seed	128, 10 epochs, seed 42 with deterministic seeding
Loss	Cross-entropy

Setting	Value
CIFAR-10	50,000 training images; the 10,000 test images serve as the validation set; 32×32 images resized to 64×64
Tiny ImageNet-200	100,000 training images (500 per class); the official 10,000 validation images (50 per class); 64×64
Augmentation (row 6 only)	RandomCrop(64, padding 4, reflect), RandomHorizontalFlip(0.5), ColorJitter(0.2, 0.2, 0.2)
Metrics	Top-1 accuracy and macro F1 on the validation set, taken at the epoch with the best validation accuracy (the checkpoint train.py saves)

Two details matter for reading the results:

- **Tiny ImageNet validation split.** The template built its Tiny ImageNet "validation" set from the training folder, which would have scored the models on images they trained on. All Tiny ImageNet results here use the official held-out validation images and labels instead.
- **Reused runs.** Rows 3 to 5 on CIFAR-10 are the Step 2 runs, and row 6 is the first 10 epochs of the Step 3 augmented run. With a constant learning rate and a fixed seed, those 10 epochs are identical to a separate 10-epoch run: the Step 2 BatchNorm run and the first 10 epochs of the Step 3 run without augmentation produced the same numbers to the last digit.
- **In-lab LeNet runs.** The in-lab milestone trained the two LeNets for 5 epochs on the CPU, reaching 52.98% (macro F1 53.12%) and 64.73% (64.44%). The master tables use new 10-epoch GPU runs with the same settings as every other row (55.63% and 66.68%), so all rows are directly comparable.

3.2 Dataset exploration

CIFAR-10 is perfectly balanced, with 5,000 training images in each of its 10 classes (imbalance ratio 1.00, Figure 3.1). Tiny ImageNet is balanced too, with 500 training and 50 validation images per class. Because no class is over-represented, accuracy and macro F1 stay within about one point of each other throughout, and no class weighting was needed.

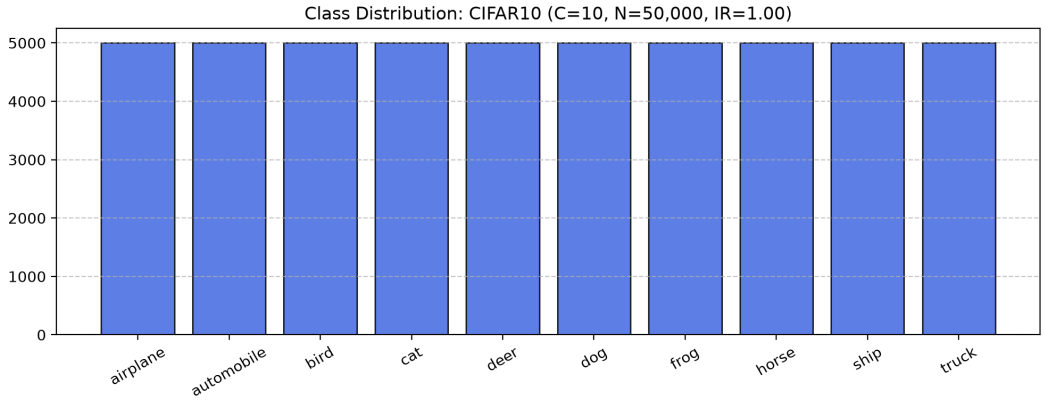


Figure 3.1: CIFAR-10 training images per class (utils/eda.py).

Pixel intensities cover the whole 0 to 1 range in all three channels (Figure 3.2). The blue channel is darker on average than red and green, and every channel has a spike at 1.0 from saturated pixels. The smaller regular spikes are a binning artifact: 256 intensity levels do not divide evenly into the histogram's bins. These statistics give the per-channel means (0.4914, 0.4822, 0.4465) and standard deviations (0.2470, 0.2435, 0.2616) that transforms.Normalize uses to give each input channel zero mean and unit variance.

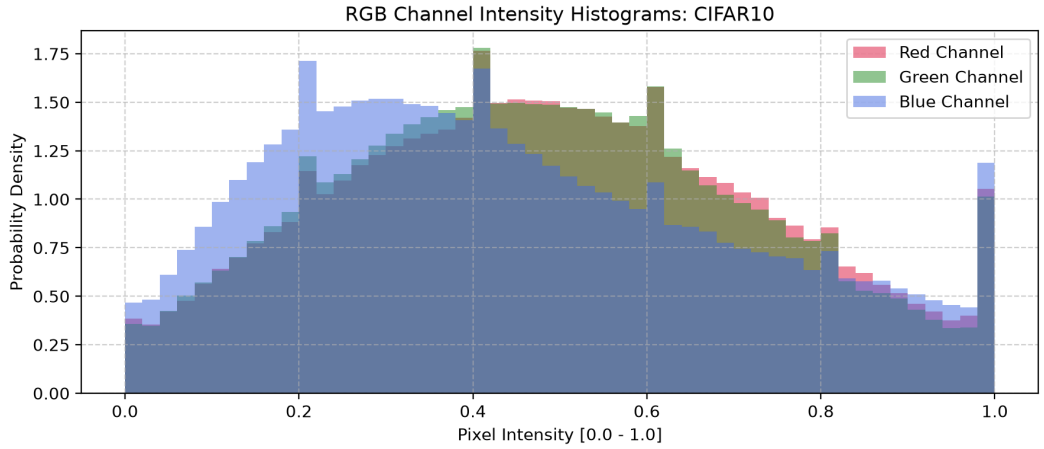


Figure 3.2: CIFAR-10 pixel intensity distribution per RGB channel (utils/eda.py).

Running `python -m utils.eda --dataset both` compares the two datasets (the README's `python utils/eda.py` form fails with a relative-import error, so the module form was used). Tiny ImageNet is just as balanced, but its images are darker, especially in blue, and more varied in every channel:

Metric	CIFAR-10	Tiny ImageNet-200
Classes	10	200
Training images	50,000	100,000
Images per class	5,000	500
Imbalance ratio (max/min)	1.00	1.00
Class entropy ratio (H/H_max)	1.0000	1.0000
RGB mean	0.4923, 0.4831, 0.4474	0.4802, 0.4481, 0.3975
RGB standard deviation	0.2404, 0.2370, 0.2557	0.2764, 0.2689, 0.2816

The CIFAR-10 statistics here are measured on the 64×64 resized images, so they differ slightly from the 32×32 values used for normalization. utils/eda.py also wrote the matching Tiny ImageNet figures (plots/eda_tiny_imagenet_*.png).

The EDA's integrity checks found CIFAR-10 clean: no corrupt files, no NaN or infinite values, no blank images, and only 1 very dark and 4 very bright images (mean intensity

below 0.05 or above 0.95) out of 50,000. The colour channels are strongly correlated (red-green 0.90, green-blue 0.90, red-blue 0.77), so much of an image's information is in brightness and shape rather than colour, which helps explain why mild colour jitter does not confuse the networks.

3.3 Data pipeline diagnostic

The same 64×64 image comes out of OpenCV and NumPy as a channels-last float32 array in host RAM, and out of torchvision as a channels-first float32 tensor on the GPU, already standardized. demo_transforms.py was extended to place the tensor on the Apple GPU (MPS) when no CUDA device exists:

```
[1] OpenCV (cv2) Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels] (Channels
-Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Color Space      : Converted BGR -> RGB
    - Memory Location  : Host System RAM (CPU)

[2] NumPy Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels] (Channels
-Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Memory Location  : Host System RAM (CPU)

[3] PyTorch (torchvision) Tensor Output:
    - Object Type      : <class 'torch.Tensor'>
    - Data Type (dtype): torch.float32
    - Shape (Layout)   : torch.Size([3, 64, 64])  --> [Channels, Height, Widt
h] (Channels-First)
    - Value Range      : Min = -1.9895, Max = 2.1265 (Standardized)
    - Memory Location  : mps:0 (Apple GPU (unified memory, MPS))
```

- **Layout:** OpenCV and NumPy store images as height × width × channels, while PyTorch convolutions expect channels × height × width, so ToTensor (or permute(2, 0, 1)) reorders the axes. OpenCV also loads colour as BGR, which must be converted to RGB before the RGB normalization statistics apply.
- **Values:** both array pipelines only rescale to 0 to 1. The torchvision pipeline also subtracts the channel means and divides by the standard deviations, giving the range of about −2 to +2 that the networks see.
- **Memory:** the array pipelines live in host RAM and need a copy to the device before training. On a discrete NVIDIA GPU that copy crosses PCIe; the M2's unified

memory lets the CPU and GPU share the same physical RAM, so the copy is cheaper but still a separate device allocation (mps:0).

The script's timing benchmark, run with the GPU otherwise idle, processes 1,000 single images through each pipeline:

```
--------------------------------------------------------------------------
---
Benchmark: Processing 1,000 Images (Resize + Flip + Normalize)
--------------------------------------------------------------------------
---
- OpenCV (CPU)           :    6.32 ms  (158,233 img/s) + Host-to-Device Copy
required
- NumPy (CPU)            :   13.42 ms  ( 74,519 img/s) + Host-to-Device Copy
required
- PyTorch (GPU Tensors)   :  599.09 ms  (  1,669 img/s) [unified memory: no h
ost-to-device copy]
```

The GPU pipeline is about 100× slower than OpenCV here because each 3×64×64 image is processed on its own: every flip and normalize is a separate GPU launch costing a fraction of a millisecond, far more than the work itself. GPU transforms only pay off on large batches, which is why train.py applies its transforms on the CPU in DataLoader workers and sends whole batches to the GPU.

3.4 Model complexity benchmark

FLOPs rise 60× from LeNet-5 to AlexNet-64, but batch-1 latency on the GPU rises only about 2×, because a single 64×64 image cannot keep the GPU busy and per-layer launch overhead dominates. Output of `python compare_models.py --dataset cifar10 --device mps --reps 200` (no CUDA GPU was available; the script was given a –device option for the Apple GPU):

```
================================================================================
ELEC 475: Lab 2 - Comprehensive Model Complexity & FLOPs Benchmark
Device        : mps (Apple Silicon GPU (MPS))
Dataset       : CIFAR10 (10 classes)
Input Shape   : (1, 3, 64, 64)
================================================================================
| Model Architecture          | Params (M) | MACs (M) | FLOPs (M) | Latency (
ms) | Throughput (img/s) |
| ----------------------------|------------|----------|-----------|----------
----|------------------ |
| 1. LeNet-5 (Base)           | 0.338      | 3.59     | 7.17      | 0.67
| 1484.9             |
| 2. LeNet-Modern (ReLU+Pool) | 0.446      | 4.59     | 9.19      | 0.59
| 1687.9             |
| 3. AlexNet-64 (None)        | 6.609      | 218.35   | 436.70    | 1.38
| 722.2              |
| 4. AlexNet-64 (LRN)         | 6.609      | 218.35   | 436.70    | 2.24
```

```
| 447.3               |
| 5. AlexNet-64 (BatchNorm)   | 6.611      | 218.98   | 437.97    | 3.57
| 280.0               |
| 6. Torchvision Zoo AlexNet  | 57.045     | 94.10    | 188.20    | 3.72
| 269.1               |
```

- **CPU for comparison:** the same benchmark on the M2's CPU gives 0.22 and 0.24 ms for the two LeNets, 2.09 ms (none), 3.41 ms (LRN) and 2.11 ms (BatchNorm) for AlexNet-64, and 6.41 ms for the Zoo AlexNet. The tiny LeNets run 2.5 to 3× faster on the CPU than on the GPU, where launch overhead outweighs their few FLOPs.
- **BatchNorm at inference:** BatchNorm adds 0.3% FLOPs and nothing measurable on the CPU (2.11 vs 2.09 ms), but its evaluation-mode kernels add 2.2 ms on the Apple GPU at batch 1. That cost is specific to the MPS backend; at deployment, BatchNorm can be folded into the preceding convolution's weights and removed entirely.
- **LRN:** thop counts LRN as zero FLOPs, yet it adds 62% to GPU latency and 63% on the CPU, because it runs as several separate memory-bound operations.
- **Zoo AlexNet:** it has the fewest FLOPs of the three AlexNets but the highest latency, because every forward pass must read its 57 M weights (228 MB in float32), most of them in the 9,216 → 4,096 FC layer.
- **Handout reference values:** the handout lists nominal scales of about 4.8, 8.2, 250 to 257 and 310 to 315 MFLOPs, and 6.31 M parameters for AlexNet-64. The measured values differ; they follow directly from the implemented layers (derived layer by layer in Section 2, counting 2 FLOPs per MAC), and the AlexNet-64 parameter count matches the template's layer specification exactly.

3.5 Master ablation tables

BatchNorm gives the largest single gain on both datasets, and the two AlexNets without normalization failed to train at all on Tiny ImageNet.

Table 7.3a: Architectural configuration and computational complexity

Model architecture	Conv depth	Normalization & regularization	Data augmentation	Trainable params (M)	FLOPs (MFLOPs)	Mean epoch time (s), CIFAR-10 / Tiny ImageNet	Inference latency (ms)
1. Base LeNet-5	2	None	None	0.338	7.17	34.2 / 43.7	0.58
2. Modernized LeNet	2	None (ReLU + overlapping MaxPool)	None	0.446	9.19	36.5 / 45.4	0.55
3. AlexNet (No Norm)	5	Dropout (p = 0.5)	None	6.609	436.70	71.6 / 116.5	1.33
4. AlexNet + LRN	5	LRN + Dropout (p = 0.5)	None	6.609	436.70	208.8 / 377.3	2.19

Model architecture	Conv depth	Normalization & regularization	Data augmentation	Trainable params (M)	FLOPs (MFLOPs)	Mean epoch time (s), CIFAR-10 / Tiny ImageNet	Inference latency (ms)
5. AlexNet + BatchNorm	5	BatchNorm2d + Dropout (p = 0.5)	None	6.611	437.97	76.8 / 126.6	3.91
6. AlexNet + BN + Aug	5	BatchNorm2d + Dropout (p = 0.5)	Pure PyTorch Aug	6.611	437.97	80.0 / 129.0	3.91
7. PyTorch Zoo AlexNet	5	PyTorch Zoo (AdaptiveAvgPool2d) + Dropout (p = 0.5)	None	57.045	188.20	74.8 / 140.8	3.41

Parameters and FLOPs are for a 64×64 input and 10 classes, from the thop profiler (2 FLOPs per MAC). Epoch time is training plus validation, averaged over the 10 epochs on the M2 GPU. Latency is the median of 300 batch-1 forward passes on the M2 GPU, taking the median of 3 repeats, measured after all training had finished; repeat measurements on different runs varied by up to about 20%, so differences under 0.5 ms are not meaningful.

Table 7.3b: Empirical predictive performance across datasets

Model architecture	CIFAR-10 val acc (%)	CIFAR-10 val macro F1 (%)	Tiny ImageNet val acc (%)	Tiny ImageNet val macro F1 (%)
1. Base LeNet-5	55.63	55.24	12.67	11.08
2. Modernized LeNet	66.68	66.85	20.56	18.85
3. AlexNet (No Norm)	74.30	73.85	0.50 (failed to train)	0.00 (failed to train)
4. AlexNet + LRN	74.27	74.30	15.21	12.50
5. AlexNet + BatchNorm	79.19	79.01	**29.30**	**27.55**
6. AlexNet + BN + Aug	**80.27**	**80.12**	28.30	26.81
7. PyTorch Zoo AlexNet	66.76	66.86	0.50 (failed to train)	0.00 (failed to train)

Each value is the best validation accuracy within 10 epochs, with the macro F1 of that same epoch; bold marks the best in each column. Random guessing scores 10% on CIFAR-10 and 0.5% on Tiny ImageNet, so rows 3 and 7 never left chance level on Tiny ImageNet (Section 4.4).

3.6 Training and validation curves

On CIFAR-10 the two LeNets overfit within 5 epochs, the unnormalized AlexNets are still improving at epoch 10, and augmentation is the only change that keeps training and validation accuracy together. Figure 3.3 shows each model's training and validation loss and accuracy side by side; Figure 3.4 overlays the validation accuracy of all seven. The per-run plots written by train.py (plots/loss_*.png) show the same curves one model at a time.

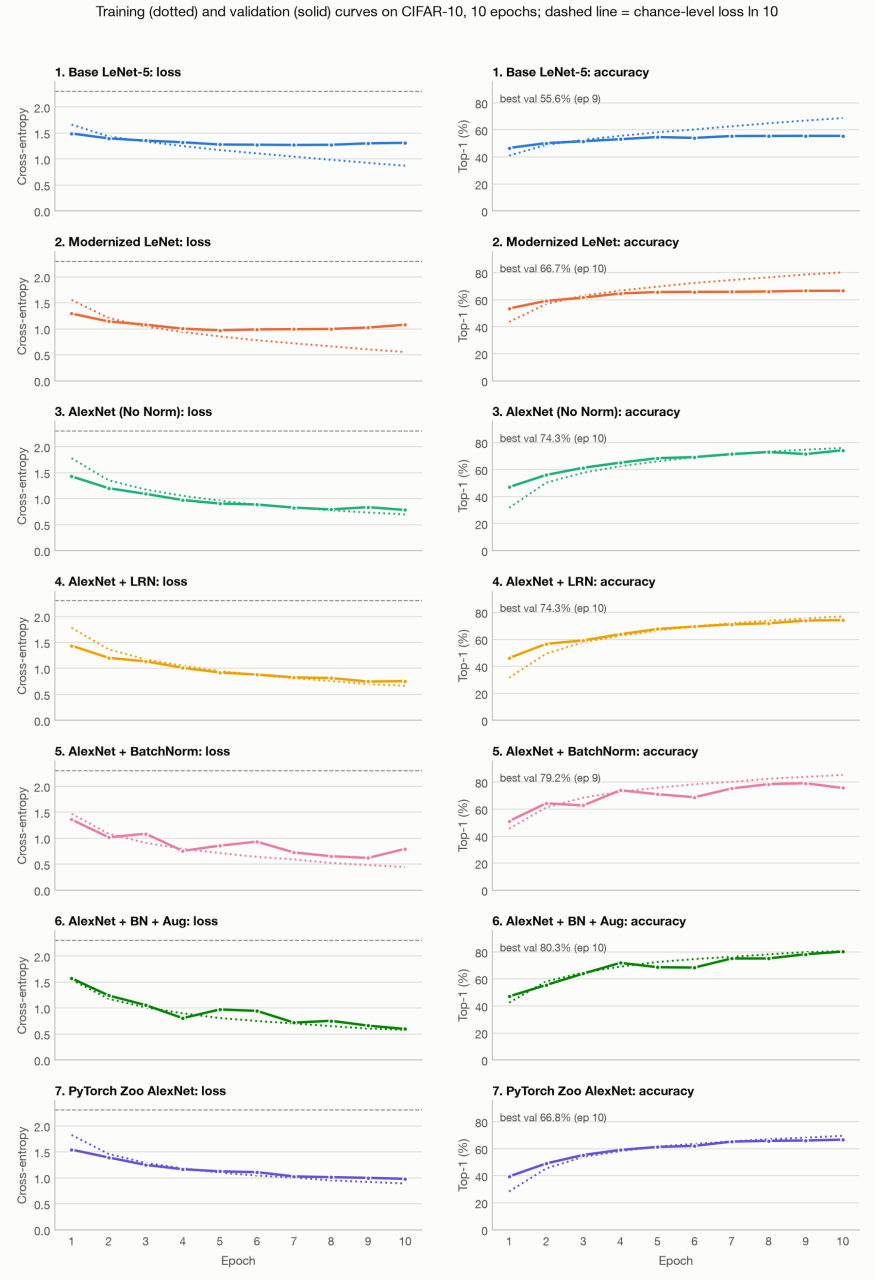


Figure 3.3: Training (dotted) and validation (solid) loss and accuracy on CIFAR-10, one row per model.

- **LeNet-5 and Modernized LeNet:** training accuracy keeps rising while validation accuracy flattens after about epoch 5, and validation loss turns upward after epoch 7 (LeNet-5) and epoch 5 (Modernized LeNet). By epoch 10 both have a 13-point gap between training and validation accuracy.
- **AlexNet without normalization and with LRN:** the two rows are nearly identical. Training and validation stay within 3 points, thanks to dropout, and both are still climbing at epoch 10.

- **AlexNet + BatchNorm:** it learns fastest (51% validation accuracy after one epoch, against 47% without normalization), but its validation curve jumps by several points from epoch to epoch, and by epoch 10 training accuracy leads by 9.7 points.
- **AlexNet + BN + Aug:** training and validation accuracy rise together (0.4-point gap at epoch 10) and reach the best result, 80.3%.
- **Zoo AlexNet:** smooth curves that level off near 67%, the same as the Modernized LeNet.

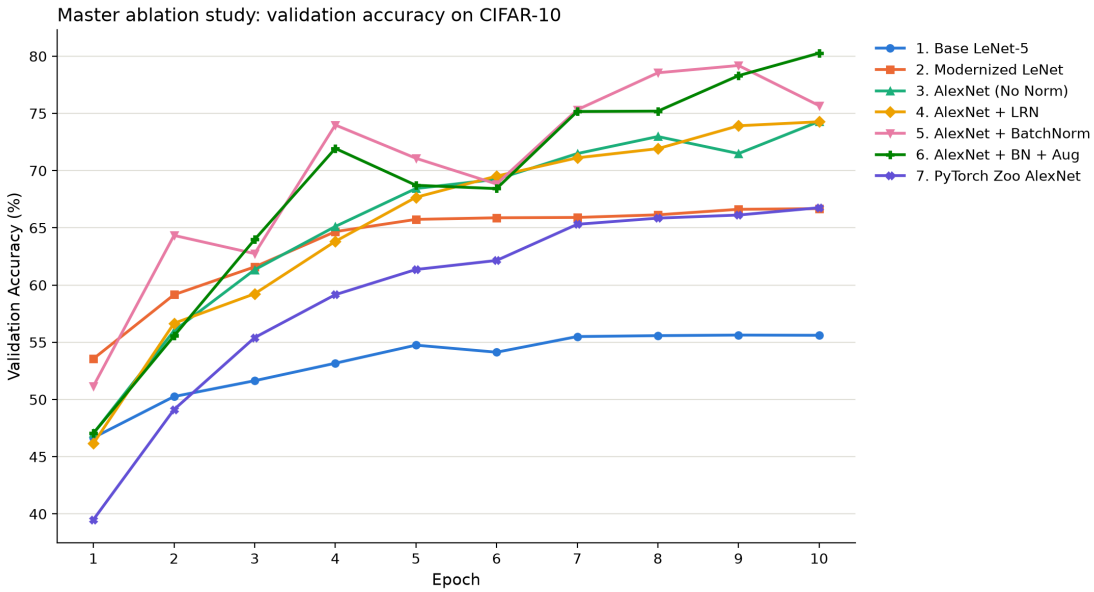


Figure 3.4: Validation accuracy per epoch for the seven ablation rows on CIFAR-10 (`python -m utils.visualization --plot-ablation`).

On Tiny ImageNet (Figures 3.5 and 3.6) every model that trained was still improving at epoch 10, and two never trained at all:

- **AlexNet without normalization and the Zoo AlexNet:** flat lines at the chance-level loss ln 200 and 0.5% accuracy for all 10 epochs (Section 4.4).
- **LeNet-5 and Modernized LeNet:** steady but slow gains to 12.7% and 20.6%, with validation loss still falling until epoch 9.
- **AlexNet + LRN:** it trained, unlike the version without normalization, but slowly, reaching 15.2%.
- **AlexNet + BatchNorm, with and without augmentation:** the fastest learners (29.3% and 28.3%), with training and validation curves close together and both still rising.

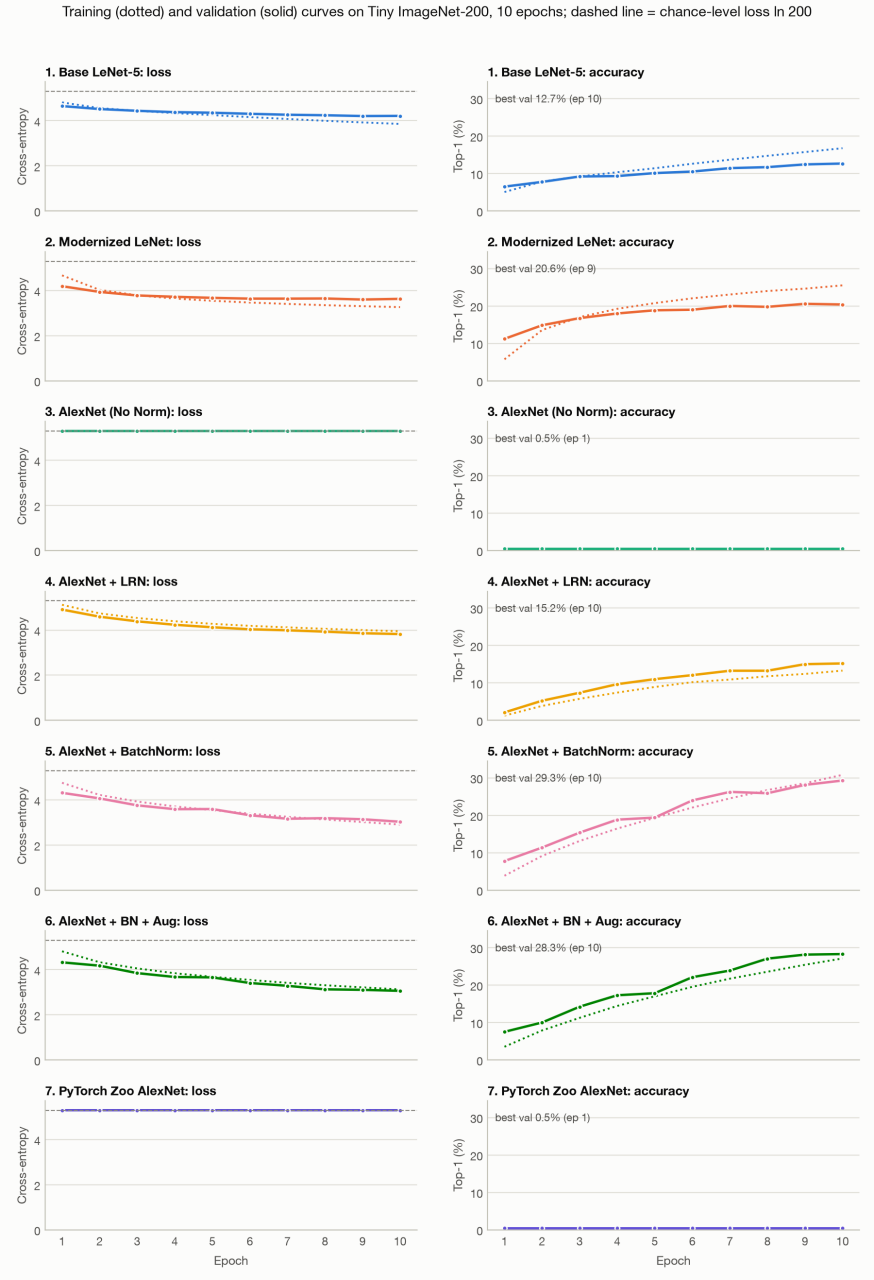


Figure 3.5: Training (dotted) and validation (solid) loss and accuracy on Tiny ImageNet, one row per model; the dashed line is the chance-level loss ln 200.

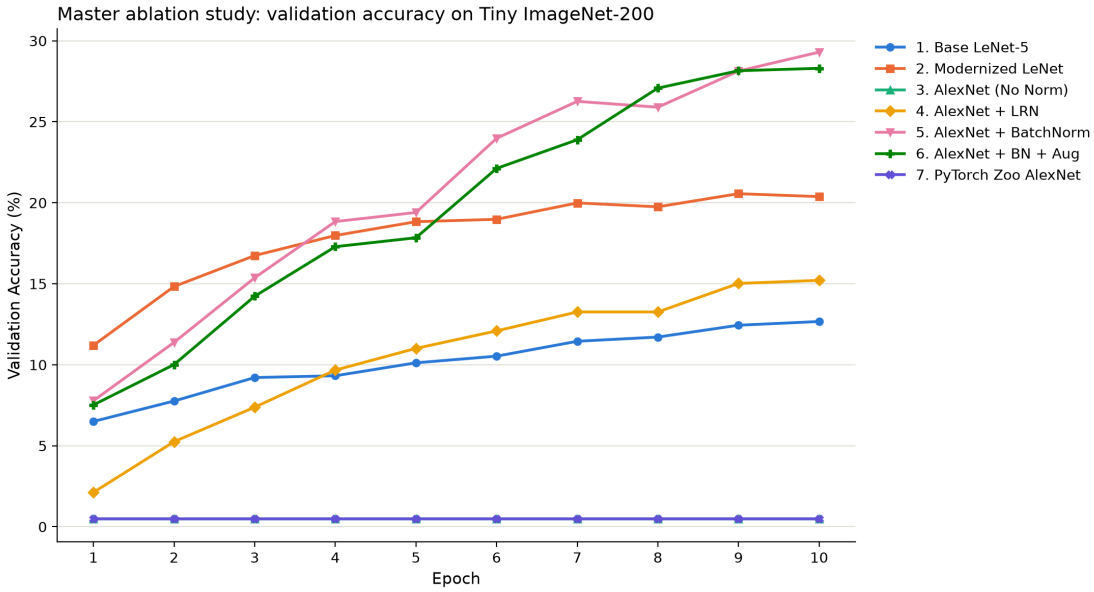


Figure 3.6: Validation accuracy per epoch for the seven ablation rows on Tiny ImageNet.

3.7 Extended ablation exploration

Beyond the mandatory ladder, five additional experiments each changed one factor against a baseline from the master tables. Training longer helped both BatchNorm models, augmentation more than halved the generalization gap, and normalization decided whether AlexNet could train on Tiny ImageNet at all.

Experiment	Baseline	Change	Metric	Baseline	Result	Δ
E1. Longer training	AlexNet + BN, CIFAR-10, 10 epochs	15 epochs	Best val acc / macro F1 (%)	79.19 / 79.01	80.55 / 80.68	+1.36 / +1.67
E2. Longer training with augmentation	AlexNet + BN + Aug, CIFAR-10, 10 epochs	15 epochs	Best val acc / macro F1 (%)	80.27 / 80.12	81.11 / 81.28	+0.84 / +1.16
E3. Augmentation, fair gap measurement	AlexNet + BN, CIFAR-10, 15 epochs	Augmentation on; gap measured on 10,000 un-augmented training images in eval mode	Generalization gap Δ at epoch 15 (points)	10.32	4.20	−6.12
E4. Normalization on Tiny ImageNet	AlexNet, no normalization (collapsed to chance)	LRN, then BatchNorm	Best val acc (%)	0.50	15.21 (LRN), 29.30 (BN)	+14.71, +28.80
E5. LRN replaced by its equivalent constant	AlexNet + LRN, one training step, batch 128, M2 GPU	Multiply by 0.595 instead of LRN (same output to within 0.1%)	Time per training step (ms)	428.9	111.9	−74%

- **E1 and E2:** neither model had converged at 10 epochs; the five extra epochs added about one point. Augmentation's benefit over 15 epochs is in E3 and Section 4.5.
- **E3:** the gap measured on clean training images is the fair comparison, because train.py's logged training accuracy is measured on augmented images with dropout active (Section 4.5).
- **E4:** the collapse and its diagnosis are in Section 4.4.
- **E5:** measured as a timing benchmark only. Because LRN's divisor was constant to within 0.1% on the trained network, the multiply gives the same outputs, so the 2.9× slowdown buys nothing.

3.8 Multi-seed evaluation

Each configuration was trained with seed 42 only; additional seeds could not be completed in the available time, so a true mean ± standard deviation across seeds is not reported. Runs are fully deterministic: the Step 2 BatchNorm run and the first 10 epochs of the Step 3 run without augmentation, both with seed 42, matched to the last digit, so repeating a seed adds no information.

As a substitute measure of how stable each result is, the table gives the mean ± sample standard deviation of validation accuracy and macro F1 over each run's final epochs. This measures epoch-to-epoch fluctuation within one run, not variation across seeds, and seed-to-seed spread is usually at least as large.

Configuration (CIFAR-10, seed 42)	Epochs averaged	Val acc (%), mean ± std	Val macro F1 (%), mean ± std
AlexNet (No Norm)	7 to 10	72.56 ± 1.35	72.17 ± 1.50
AlexNet + LRN	7 to 10	72.81 ± 1.53	72.75 ± 1.70
AlexNet + BatchNorm	7 to 10	77.18 ± 1.97	77.38 ± 1.59
AlexNet + BatchNorm (15-epoch run)	11 to 15	77.40 ± 4.34	77.54 ± 4.16
AlexNet + BN + Aug (15-epoch run)	11 to 15	79.62 ± 1.14	79.40 ± 1.29

- **Clear differences:** BatchNorm's lead over no normalization and LRN (about 4.5 points) is more than twice the fluctuation of any of the three, so it is unlikely to be a seed effect.
- **Unclear differences:** no normalization and LRN differ by 0.25 points, far inside their fluctuation of about 1.4 points. Augmentation's 2.2-point lead in the 15-epoch runs is about half the BatchNorm-only run's fluctuation of 4.3 points, so it needs more seeds to confirm; it did reduce that fluctuation almost fourfold.
- **Reproducing with seeds:** `run_step5_extra.sh` in the template folder trains the BatchNorm configurations with seeds 1 and 2 (about 13 minutes per run) for a full mean ± std across three seeds.

4. Technical Analysis and Discussion

4.1 Overview of the ablation ladder

Three changes produced most of the gains: modern activations and pooling (+11.1 points on CIFAR-10), depth (+7.6) and BatchNorm (+4.9). LRN added nothing, and the Zoo AlexNet fell 7.5 points below the AlexNet-64 it was meant to benchmark. On Tiny ImageNet, BatchNorm decided whether a deep network trained at all. Each row of the table changes one thing from the row it compares against:

Step	What changed	Δ CIFAR-10 val acc (points)	Δ Tiny ImageNet val acc (points)	Cost
LeNet-5 → Modernized LeNet	tanh → ReLU, 2×2 average → 3×3 overlapping max pooling, same padding	+11.05	+7.89	+28% FLOPs, +7% epoch time
Modernized LeNet → AlexNet (no norm)	5 conv stages (64 to 384 filters), dropout	+7.62	−20.06 (training collapsed)	47.5× FLOPs, 2.0× epoch time
AlexNet → + LRN	LRN after stages 1 and 2	−0.03	+14.71 (the version without normalization had collapsed)	2.9× epoch time
AlexNet → + BatchNorm	BatchNorm in all 5 stages	+4.89	+28.80 (from chance)	+7% epoch time
+ BatchNorm → + augmentation	crop, flip, colour jitter	+1.08	−1.00	+4% epoch time
AlexNet-64 (no norm) → Zoo AlexNet	torchvision AlexNet at 64×64	−7.54	0.00 (both collapsed)	8.6× parameters, 0.43× FLOPs

The sections below explain each step, then compare the datasets and the costs.

4.2 Activation and pooling: LeNet-5 vs Modernized LeNet

Replacing tanh and average pooling raised CIFAR-10 accuracy from 55.6% to 66.7% and Tiny ImageNet accuracy from 12.7% to 20.6%, for 28% more FLOPs. The Modernized LeNet also learns faster: 53.6% validation accuracy after one epoch against 46.7%, and a final training loss of 0.55 against 0.87.

- **ReLU keeps gradients alive.** The derivative of tanh, $1 - \tanh^2(x)$, is at most 1 and falls toward 0 once $|x|$ exceeds about 2, so saturated units pass back almost no gradient, and the factors multiply across layers. ReLU passes a gradient of exactly 1 for every positive input, so each weight update is larger and better directed. ReLU also outputs exact zeros for negative inputs, giving sparse features that are cheaper to separate linearly.
- **Max pooling keeps the strongest response.** A convolution filter signals that its pattern is present; averaging a 2×2 window dilutes that signal with weaker neighbours, while max pooling passes the strongest match through and tolerates

small shifts of it. Overlapping 3×3 windows with stride 2 sample every position at least once with some redundancy, which reduces the aliasing of non-overlapping pooling and widens the receptive field from 16 to 19 pixels.

- **A confound.** Padding 2 also keeps the feature maps larger (15×15 instead of 13×13 before the classifier), giving FC1 3,600 inputs instead of 2,704 and 107,520 more parameters. Part of the gain may come from that extra spatial detail rather than from the activation and pooling alone.
- **Both still overfit.** With 96 to 97% of their parameters in FC1 and no dropout, both LeNets end with a 13-point gap between training and validation accuracy, and their validation loss starts rising after epoch 7 (LeNet-5) and epoch 5 (Modernized LeNet).

4.3 Depth and capacity: Modernized LeNet vs AlexNet-64

Going from two to five convolution stages raised CIFAR-10 accuracy from 66.7% to 74.3% (macro F1 66.9% to 73.9%), and the deeper network overfits far less despite having 15× more parameters.

- **Richer features.** Each extra stage builds on the last, from edges and colour blobs to textures and object parts. The receptive field grows from 19 pixels in the Modernized LeNet to 97 pixels in AlexNet-64, so the final features can respond to whole objects rather than fragments.
- **Capacity without memorization.** The training-validation gap fell from 13.6 to 1.7 points. Dropout ($p = 0.5$) before the two hidden FC layers stops units from co-adapting, and only 52% of AlexNet-64's parameters sit in FC layers, against 99% in the LeNets; convolutional weights are shared across positions, so they add capacity that generalizes instead of memorizing.
- **Still undertrained.** AlexNet-64's validation loss was still falling at epoch 10 (its lowest value came at the last epoch), so a longer schedule would likely widen its lead.
- **Cost.** FLOPs grow 47.5× but epoch time only 2.0× (36.5 s to 71.6 s): the LeNets are limited by image loading on the CPU, and the larger network keeps the GPU busier.
- **Fragility.** The same unnormalized network failed to train at all on Tiny ImageNet (Section 4.4), so depth alone also made optimization harder.

4.4 Normalization: none vs LRN vs BatchNorm

BatchNorm raised CIFAR-10 accuracy by 4.9 points (74.3% to 79.2%) for 7% more training time, while LRN changed nothing and nearly tripled the epoch time. On Tiny ImageNet the difference was larger still: without normalization AlexNet never left chance level, while the BatchNorm version reached 29.3%.

Answers to the Step 2 questions.

- **Performance improvement: Yes.** BatchNorm raised best validation accuracy to 79.19% and macro F1 to 79.01%, against 74.30% / 73.85% without normalization and 74.27% / 74.30% with LRN (+4.9 points accuracy, +4.7 to +5.2 points F1).
- **Computational overhead: wall-clock time Yes, FLOPs No.** Epoch time rose 2.9× with LRN (208.8 s) and 1.07× with BatchNorm (76.8 s) against 71.6 s, while FLOPs rose only 0.3% for BatchNorm and 0% as counted (about 0.25% by hand) for LRN.
- **Most stable gradient progression: BatchNorm.** Its training loss fell fastest with the smallest spikes; per-batch noise was similar for all three, and its validation curve was the noisiest (details below).

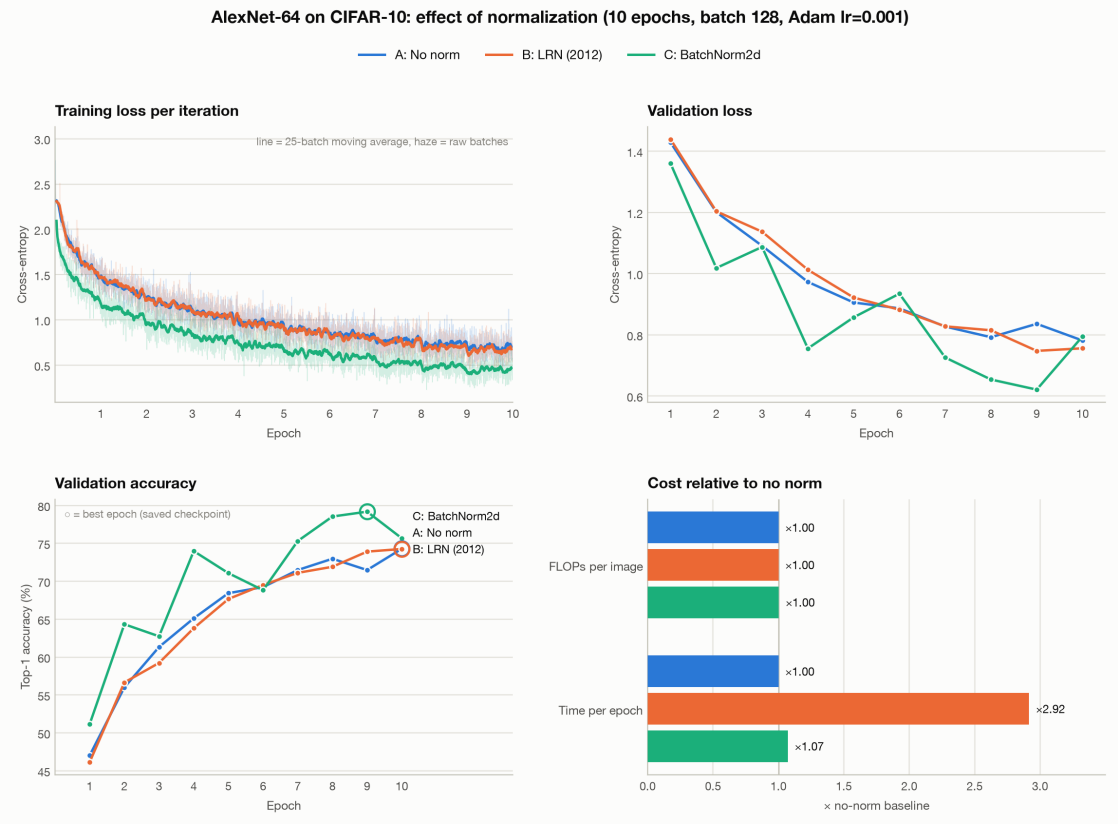


Figure 4.1: Training loss per iteration, validation loss, validation accuracy and cost relative to no normalization for the three AlexNet-64 variants on CIFAR-10.

Why BatchNorm helps. BatchNorm standardizes each channel over the mini-batch, $\hat{x} = (x - \mu_B) / \sqrt{(\sigma_B^2 + \varepsilon)}$, then rescales it with learned γ and β (Section 2.4).

- **Stable layer inputs:** every layer receives inputs with a fixed mean and variance however much the earlier weights move, which reduces internal covariate shift, so later layers stop chasing a moving input distribution.
- **Smoother loss surface:** the gradient that flows back through BatchNorm is scaled by γ/σ_B and loses its components along the batch mean and along $\hat{x}$ (Santurkar et

al., 2018). That bounds its size and keeps it predictive over a longer step, so the optimizer can move further per update.

- **Scale invariance:** BN(a·Wx) = BN(Wx) for any a > 0, so activations cannot blow up or die out across the five stages.
- **Evidence:** after 100 batches BatchNorm's training loss was 1.57, against 1.92 without normalization and 1.88 with LRN. It passed 72% validation accuracy at epoch 4, where the other two needed epochs 8 and 9, and averaged over epochs 7 to 10 it scored 77.2% against 72.6% and 72.8%.

Why LRN did nothing. Measured on the trained network, LRN's divisor $(k + (\alpha/n)\Sigma a^2)^{\beta}$ equals 1.6818 = 2^0.75 for every activation, varying by less than 0.1%. With standardized inputs, $(\alpha/n)\Sigma a^2$ never exceeds about 0.002, far below k = 2, so LRN reduces to multiplying stages 1 and 2 by a constant 0.595, which the next convolution absorbs. Its constants were tuned for the 2012 network's raw-pixel ImageNet inputs, whose activations were much larger.

Stability. BatchNorm's training loss falls fastest and has the smallest spikes, but batch-to-batch noise is similar for all three (standard deviation around the trend 0.087, 0.096 and 0.093 over epochs 6 to 10), since most of it comes from mini-batch sampling. BatchNorm's validation curve is the noisiest: its validation loss rose in 4 of 9 epochs, against 1 of 9 for the others, because evaluation uses running averages of μ_B and σ_B^2 that lag behind weights still moving at a constant learning rate.

Cost. BatchNorm adds 0.3% FLOPs and 7% epoch time. LRN adds no counted FLOPs but makes each epoch 2.9× slower: PyTorch builds it from about seven separate memory-bound operations, and on the M2 GPU one of them, the 5-D padding of the 192-channel layer, takes 281 ms of LRN's 293 ms forward and backward pass per batch, against 4 ms for BatchNorm on the same tensor.

Tiny ImageNet: training collapse without normalization. The unnormalized AlexNet's training loss spiked to 5.50 at the third batch, then locked at ln 200 = 5.298 by the fifth and stayed there for all 7,810 steps, with validation accuracy at chance (0.50%).

- **Diagnosis:** the saved network gives exactly the same output for every input image. Between 95% and 99% of the ReLU outputs in Conv5 and the FC layers are zero, so almost no gradient reaches the earlier layers, and the last layer's bias can only learn the uniform class prior, whose loss is exactly ln 200. The Zoo AlexNet failed the same way, with 100% of Conv4's ReLU outputs at zero.
- **CIFAR-10 contrast:** both networks also spiked in their first steps on CIFAR-10 but recovered within about 40 steps; on Tiny ImageNet they never did.
- **Why BatchNorm survives:** it re-centres every channel on each batch, so roughly half the pre-activations stay positive whatever the scale of the weights, and the network cannot fall into the all-zero state. The BatchNorm version trained normally to 29.3%.

- **Likely trigger (not tested):** Adam's first updates move every weight by about the learning rate, whatever the gradient's size, and in a deep unnormalized network one such step can push most pre-activations negative at once. A lower learning rate or a warm-up period would probably avoid the collapse.

LRN on Tiny ImageNet. The LRN network did train, reaching 15.2% (macro F1 12.5%), but only half as well as BatchNorm (29.3%), with a final training loss of 3.95 against 2.91, and each epoch took 377 s, 3.0× BatchNorm's 127 s. Since LRN here amounts to scaling stages 1 and 2 by a constant 0.595, its escape from the collapse suggests the collapse depends on how large the early activations are, though with one seed per run it could also be chance. Either way, LRN only avoided the failure; BatchNorm is what made the network train well.

4.5 Data augmentation and the generalization gap

Augmentation cut the generalization gap by more than half but moved accuracy little within these short schedules: +1.1 points on CIFAR-10 at 10 epochs and −1.0 on Tiny ImageNet.

Answers to the Step 3 questions (15-epoch runs, epoch 15).

Run	Train acc, logged (%)	Train acc, clean (%)	Val acc (%)	Δ logged (points)	Δ clean (points)	Val macro F1 (%)
No augmentation	90.51	90.87	80.55	9.96	10.32	80.68
With augmentation	84.63	82.60	78.40	6.23	4.20	77.85

- **Generalization gap reduction: Yes.** Δ fell from 10.32 to 4.20 points measured on clean training images (9.96 to 6.23 using train.py's logged training accuracy).
- **Validation metric improvement: Marginal.** At epoch 15 augmentation was lower (−2.15 points accuracy, −2.82 F1), at the best epoch slightly higher (+0.56 / +0.60), and averaged over epochs 11 to 15 higher by 2.2 points with a 4× steadier curve. The reasons are discussed below.

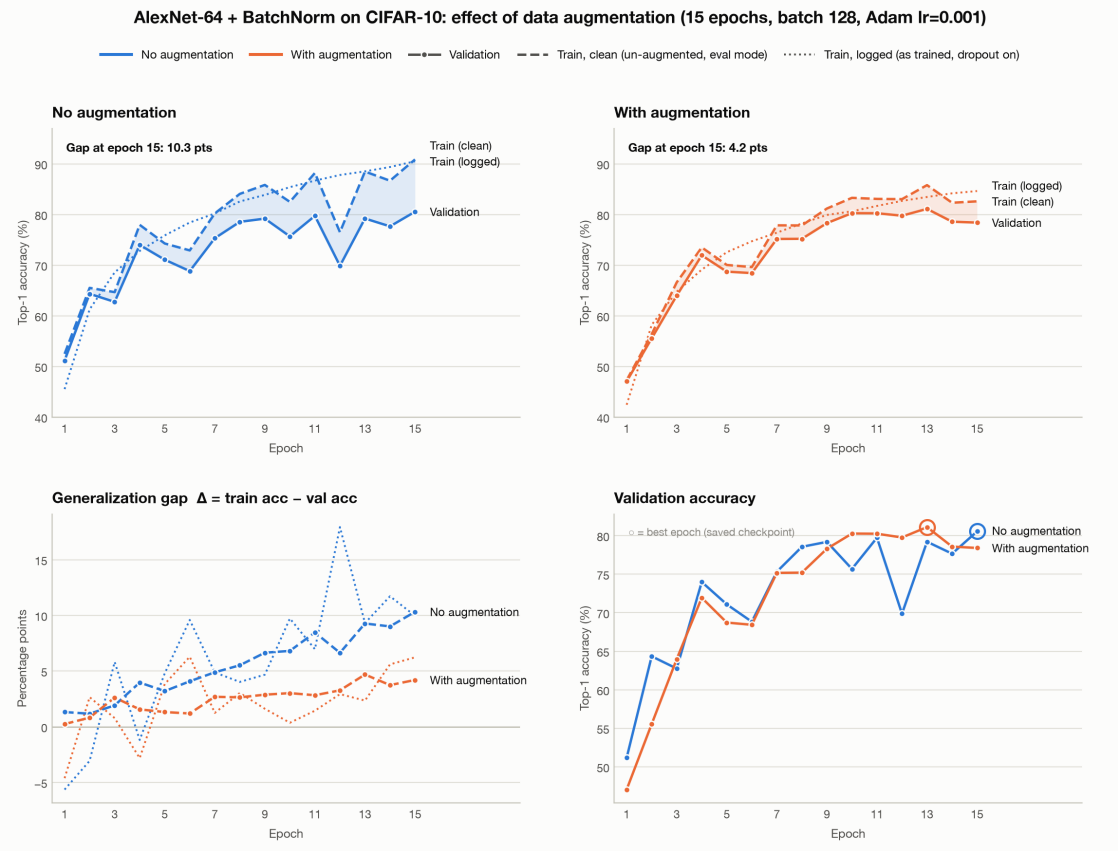


Figure 4.2: Training and validation accuracy with and without augmentation (shaded band = gap), the gap per epoch, and validation accuracy, for the 15-epoch runs of Step 3.

The gap. In the 15-epoch runs of Step 3, the gap Δ = training accuracy − validation accuracy was measured on 10,000 un-augmented training images in evaluation mode, the same way as validation. At epoch 15 it was 10.3 points without augmentation and 4.2 with it; averaged over epochs 11 to 15 it was 8.7 against 3.7. Without augmentation the gap grew steadily, by about 0.65 points per epoch, which is memorization building up.

Why augmentation shrinks it. Each training image gets a fresh random crop, flip and colour jitter every time it is used, so training minimizes the average loss over all those versions, E_T[L(f(T(x)), y)], and the same pixels almost never repeat. Memorizing exact pixel arrangements stops paying off; only features that survive the transforms reduce the loss.

- **Random crops** shift the object. Convolutions handle shifts by themselves, but the FC layers read a 3×3×256 map in which position matters, so shifts teach them not to rely on where a feature appears.
- **Horizontal flips** leave every CIFAR-10 class unchanged, so filters must recognize patterns in either orientation.
- **Colour jitter** varies brightness, contrast and saturation by up to 20%, so the network cannot rely on absolute colour (“blue background means ship”) and must use shape and texture.

Why accuracy barely moved.

- **Short schedule:** at epoch 15 the augmented model's training loss was still falling by 0.02 per epoch. The model without augmentation had not yet paid for memorizing: its validation loss bottomed out at epoch 11, but its accuracy was still at its peak at epoch 15.
- **Noise larger than the effect:** BatchNorm's evaluation-mode statistics make validation accuracy swing by 3 to 5 points between epochs. The dips also appear in the clean training accuracy (correlation 0.99 between their epoch-to-epoch changes), so they are not overfitting. Over epochs 11 to 15, augmentation was ahead by 2.2 points on average (79.6% against 77.4%) and 4× steadier (standard deviation 1.0 against 3.9 points), even though it was 2.2 points behind at epoch 15 alone.
- **Not too severe:** the transforms are mild (4 pixels of padding on a 64-pixel image upsampled from 32 is a shift of about 2 original pixels), and in the last five epochs the model scored within 2 points on augmented and clean training images.

Tiny ImageNet. Without augmentation, the BatchNorm model's gap on Tiny ImageNet was only 1.5 points at epoch 10, so it was not yet overfitting. Augmentation then regularized a model that was still underfitting, which cost a point (29.3% to 28.3%).

4.6 PyTorch Zoo AlexNet vs AlexNet-64

With 57 M parameters, torchvision's AlexNet reached only 66.8% on CIFAR-10: 7.5 points below the 6.6 M-parameter AlexNet-64 and level with the 0.45 M-parameter Modernized LeNet. On Tiny ImageNet it never trained (Section 4.4). The architecture, not training, is the limit.

- **Built for 224×224 inputs.** Its first layer, 11×11 filters with stride 4, shrinks a 64×64 image to 15×15 at once (AlexNet-64 keeps 32×32), discarding much of the fine detail a small image has. By Conv3 the maps are 3×3, and the last pooling layer leaves a single 1×1 value per channel.
- **Parameters that carry no information.** AdaptiveAvgPool2d copies that single value into all 36 cells of its 6×6 output, so FC1's 37.7 M weights (66% of the model) act on only 256 distinct numbers, and all spatial layout is gone. The classifier is huge but has almost nothing to work with.
- **Not an overfitting problem.** Its training-validation gap was only 2.9 points thanks to dropout, and its validation accuracy levelled off near 67%: the features, not the regularization, limit it.
- **No normalization.** Like AlexNet-64 without BatchNorm, it collapsed to chance on Tiny ImageNet.
- **Cost.** It has the fewest FLOPs of the AlexNets (188 against 437 MFLOPs) but 2.6× the inference latency of AlexNet-64 without normalization (3.41 against 1.33 ms) and slower epochs (74.8 against 71.6 s on CIFAR-10, 140.8 against 116.5 s on Tiny

ImageNet), because every forward pass reads, and every Adam step updates, 57 M weights.

Using it as intended would mean resizing inputs to 224×224, which raises its cost to 1,420 MFLOPs per image (3.3× AlexNet-64), or starting from its ImageNet weights with – pretrained; neither was tested here. AlexNet-64's smaller first stride and narrower FC layers are what make the design fit 64×64 data.

4.7 CIFAR-10 vs Tiny ImageNet

AlexNet-64 learned real features on Tiny ImageNet, reaching 28.3% top-1, 57× the 0.5% chance rate, but that is barely a third of the 80.3% it reaches on CIFAR-10 with identical architecture, training and augmentation.

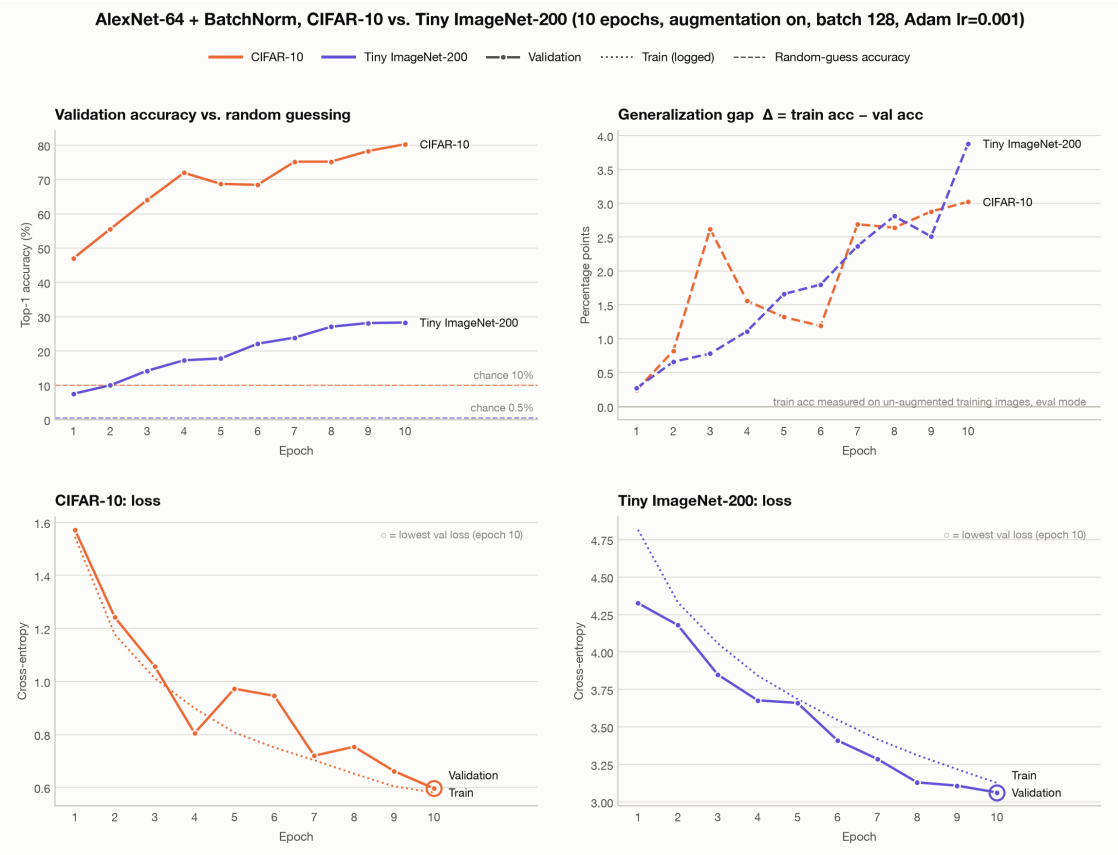


Figure 4.3: Validation accuracy against random guessing, generalization gap, and training and validation loss for the same model and settings on both datasets (row 6 of the master tables).

Generalization above chance: Yes. By chance the model would get 50 of the 10,000 validation images right, with a standard deviation of about 7 images; it got 2,830 right, around 390 standard deviations above chance. Its macro F1 of 26.8% shows the learning spans many classes, not a few easy ones. Every model that trained also cleared chance

easily: 12.7% for LeNet-5, 20.6% for the Modernized LeNet and 29.3% for AlexNet + BatchNorm.

Cross-dataset comparison.

Dataset (row 6)	Classes	Training images per class	Val acc (%)	Macro F1 (%)	Accuracy / chance
CIFAR-10	10	5,000	80.27	80.12	8.0×
Tiny ImageNet-200	200	500	28.30	26.81	56.6×

On CIFAR-10 accuracy and macro F1 are within 0.2 points; on Tiny ImageNet F1 trails accuracy by 1.5 points, so performance is less even across classes. Tiny ImageNet is much harder for four reasons:

- **Many fine-grained classes.** Among its 200 classes are six dog breeds (Chihuahua, Yorkshire terrier, golden retriever, Labrador retriever, German shepherd, standard poodle), three kinds of cat (tabby, Persian, Egyptian) and several similar vehicles (sports car, beach wagon, school bus, trolleybus), separated by small details rather than overall shape.
- **Ten times less data per class.** 500 training images per class against 5,000 leaves each decision boundary far less evidence.
- **Harder images.** The photos are downsampled from ImageNet, with objects that are often small, partly cut off or set against cluttered backgrounds. They hold more real detail than CIFAR-10's upsampled 32×32 images, but also much more that is irrelevant.
- **A higher bar.** The same 1,024 features must separate 200 classes instead of 10, and the starting loss is ln 200 = 5.30 instead of ln 10 = 2.30.

Overfitting trajectory: the opposite of an earlier divergence. On Tiny ImageNet the validation loss neither diverged nor plateaued within 10 epochs; it was still falling and at its lowest at epoch 10 in both BatchNorm runs (3.033 without augmentation, 3.063 with). On CIFAR-10, the same model without augmentation had its lowest validation loss at epoch 9 and jumped at epoch 10 (0.621 to 0.795), and the LeNets' CIFAR-10 validation loss turned upward after epochs 5 to 7, while on Tiny ImageNet theirs kept falling until epoch 9. The training and validation losses on Tiny ImageNet fall together (Figure 4.3, bottom right), and the gap stays small (1.5 points without augmentation; 3.9 points with augmentation, measured on clean training images). The model cannot yet fit even its training set (31% training accuracy), so memorization, which is what drives validation loss upward, has not begun. With only 500 images per class, overfitting would be expected to set in over a longer schedule.

4.8 Computational cost vs accuracy

FLOP counts predicted neither accuracy nor running time well; what each layer does, and how well it suits the hardware, mattered more.

- **FLOPs vs accuracy.** The Zoo AlexNet spends 188 MFLOPs per image to match the 66.7% that the Modernized LeNet reaches with 9.2 MFLOPs, while AlexNet-64 turns 437 MFLOPs into 74% to 80%. Where the computation goes (spatial resolution kept early, features that see the whole image) counts for more than the total.
- **FLOPs vs epoch time.** The LeNets need 47× fewer FLOPs than AlexNet-64 but train only 2× faster (34 to 37 s per CIFAR-10 epoch against 72 s), because their speed is set by the CPU preparing images, not by the GPU. LRN adds no counted FLOPs but triples the epoch time, and BatchNorm's 0.3% extra FLOPs cost 7% more time.
- **Dataset size.** Tiny ImageNet has twice as many training images, and its epochs took 1.3× (LeNet-5) to 1.9× (Zoo AlexNet) as long as on CIFAR-10; the image-loading-bound LeNets slow down least, probably because Tiny ImageNet images are already 64×64 and skip CIFAR-10's resize step.
- **Latency.** At batch size 1 the GPU is mostly idle, so latency tracks the number of layers and launches more than FLOPs: 0.6 ms for the LeNets, 1.3 ms for AlexNet-64, and 3.4 ms for the Zoo AlexNet, which must read 228 MB of weights per image. BatchNorm's 3.3 to 3.9 ms on the M2 GPU (against 2.1 ms on its CPU, the same as no normalization) is a weakness of the MPS backend, and folding BatchNorm into the convolution weights would remove it.
- **Value for cost.** BatchNorm is the best trade in the study (+4.9 points for 7% more training time), augmentation is close behind on CIFAR-10 (+1.1 points for 4%), and LRN is the worst (no gain for 2.9× the time). The best model, AlexNet + BN + Aug, reached 80.3% in about 13 minutes of training.

4.9 Limitations

The large effects in this study (activation and pooling, depth, BatchNorm, the dataset gap, the training collapse) are well beyond run-to-run noise; the small ones (LRN against none, augmentation against none) are not settled by these runs.

- **One seed per configuration.** BatchNorm models' validation accuracy moves by 3 to 5 points between epochs, so differences of a few points between single runs are within noise.
- **Short, fixed schedule.** Ten epochs at a constant learning rate leave the AlexNets still improving. A longer schedule with learning-rate decay would raise every deep model and would likely show more benefit from augmentation.
- **The collapse was seen once.** The unnormalized networks failed on Tiny ImageNet with one seed and one learning rate (10^{-3}); whether a lower rate or a warm-up avoids it was not tested.
- **Hardware-specific timings.** All times were measured on an M2 GPU through PyTorch's MPS backend, whose slow paths (LRN's padding, BatchNorm at batch 1) inflate some costs. Absolute times will differ on a CUDA GPU, and the FLOP counts differ from the handout's reference values (Section 3.4).

- **The Zoo AlexNet was used outside its design.** Training it from scratch on 64×64 inputs measures how poorly it transfers to small images, not how well it performs at 224×224 with ImageNet weights.
- **Validation used twice.** The reported accuracy is the best epoch on the same validation set used to pick that epoch, which flatters every model slightly; a separate test set would remove that bias.